U.PORTO FEUP FACULDADE DE ENGENHARIA UNIVERSIDADE DO PORTO
DEI DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

Analysis and Synthesis of Algorithms
Design of Algorithms

Integer Linear Programming

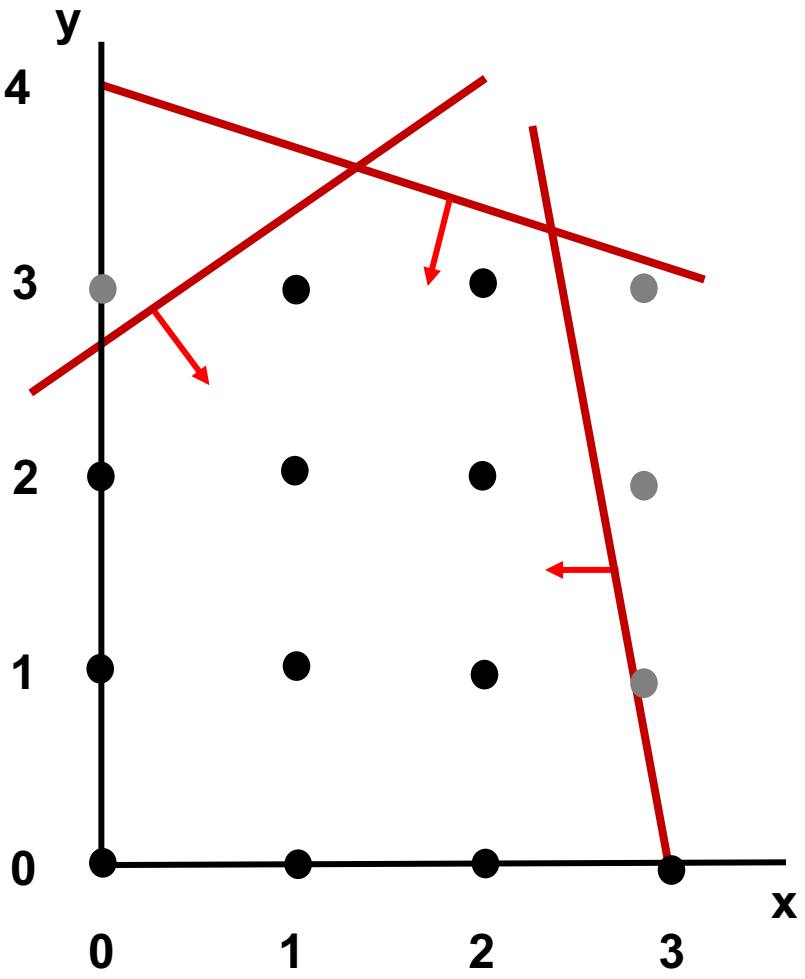
Complexity and Algorithmic Approaches

Outline

- Integer Linear Programming (ILP)
- Brute-Force and Naive Approaches
- Complexity
- The Cutting-Plane Method
- Branch-and-Bound Search

Integer Linear Programming (ILP)

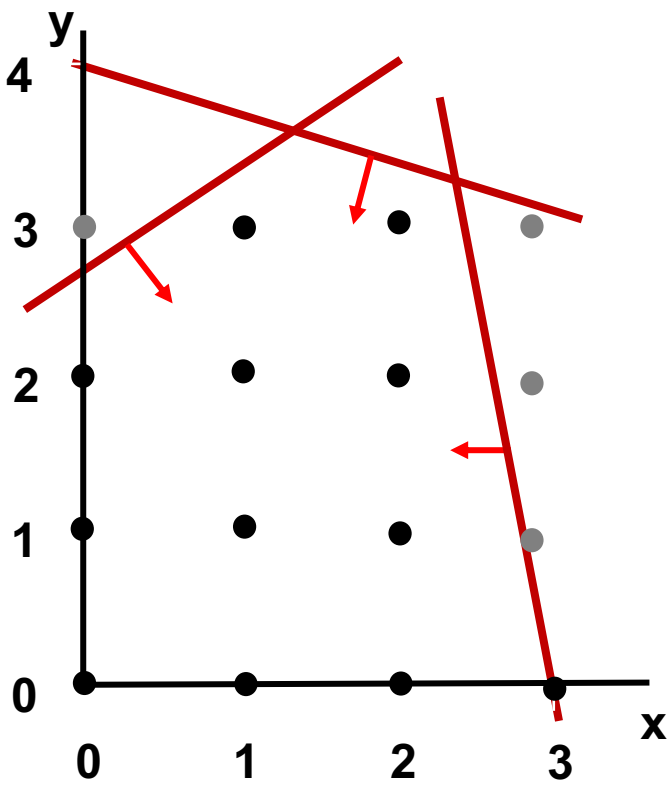
- Feasible Region: Set of discrete points (bounded by a convex region)
- Optimal Solution may not be a corner point.



How to Approach it?

A Brute-Force Algorithm for ILP

- Simple Algorithm:
  - Enumerate all Integer Points in Convex Feasible
  - Evaluate the Cost Function
  - Pick the "Best"
- Problem:
  - N Constraints $\Rightarrow 2^N$ Points



ILP is NP-Complete

Claim: ILP is NP-Complete

Proof:

(1) ILP is in NP.

(2) Reduction of 3-CNF-SAT to ILP.

(1) ILP is in NP

For the decision version of ILP (proving that there exists an integer vector that meets the matricial inequality constraint) we just "guess" the correct assignment of values to $\vec{x}$, evaluate and check if the constraints are met.

ILP is NP-Complete

(2) Reduction of 3-CNF-SAT to ILP:

Let the variables in the 3-CNF-SAT formula be x_1, x_2, ..., x_n. we define the corresponding variables z_1, z_2, ..., z_n in our integer linear program as:

First, restrict each variable to be 0 or 1: $0 \leq z_i \leq 1 \quad \forall\, i$

Assigning z_i=1 in the integer program represents setting x_i=true in the formula, and assigning z_i=0 represents setting x_i=false.

For each clause like (x_1 OR $\neg x_2$ OR x_3), have a constraint like:

$z_1 + (1-z_2) + z_3 > 0$.

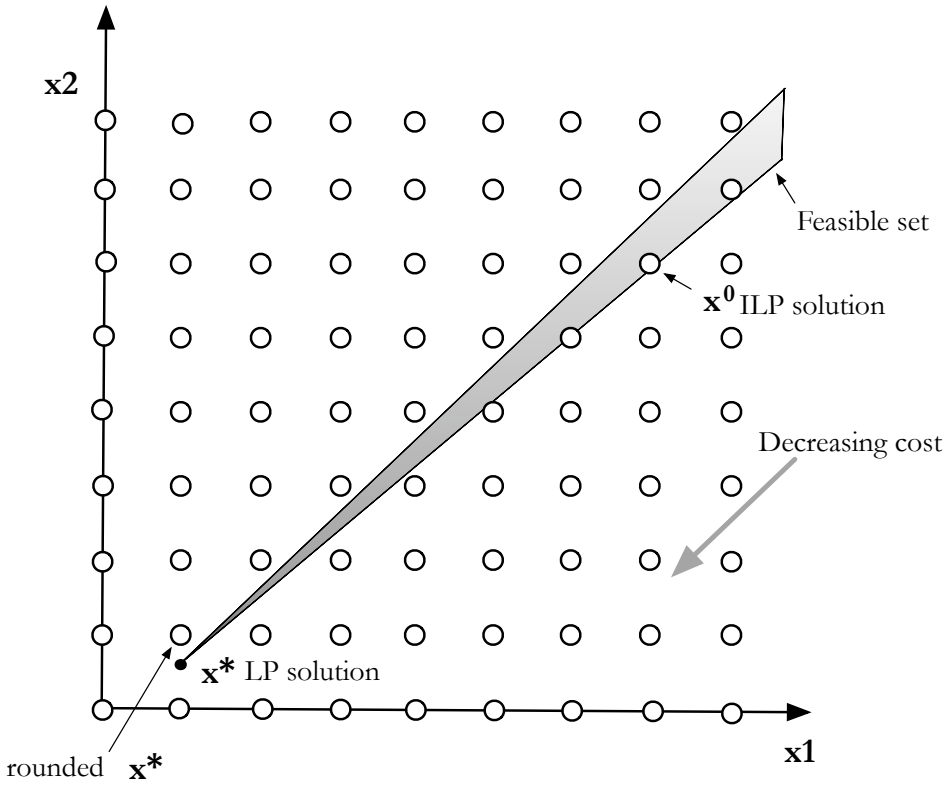
To satisfy this inequality we must either set z_1=1 or z_2=0 or z_3=1, which means we either set x_1=true or x_2=false or x_3=true in the corresponding truth assignment.

A Naive Approach to ILP

- Using LP to Solve ILP?
  - If Solution is Integer, we're done.
  - If Not, just "round it off" to the nearest integer.

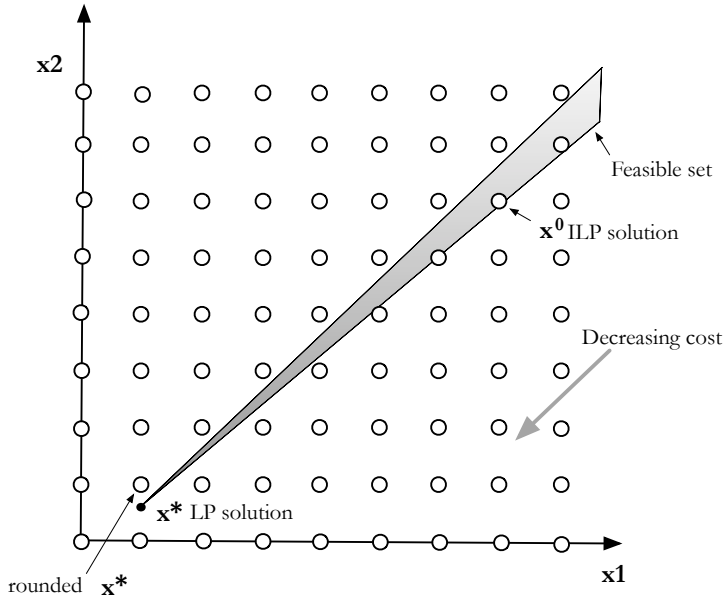
A Naive Approach to ILP

- Using LP to Solve ILP?
  - If Solution is Integer, we're done.
  - If Not, just "round it off" to the nearest integer.



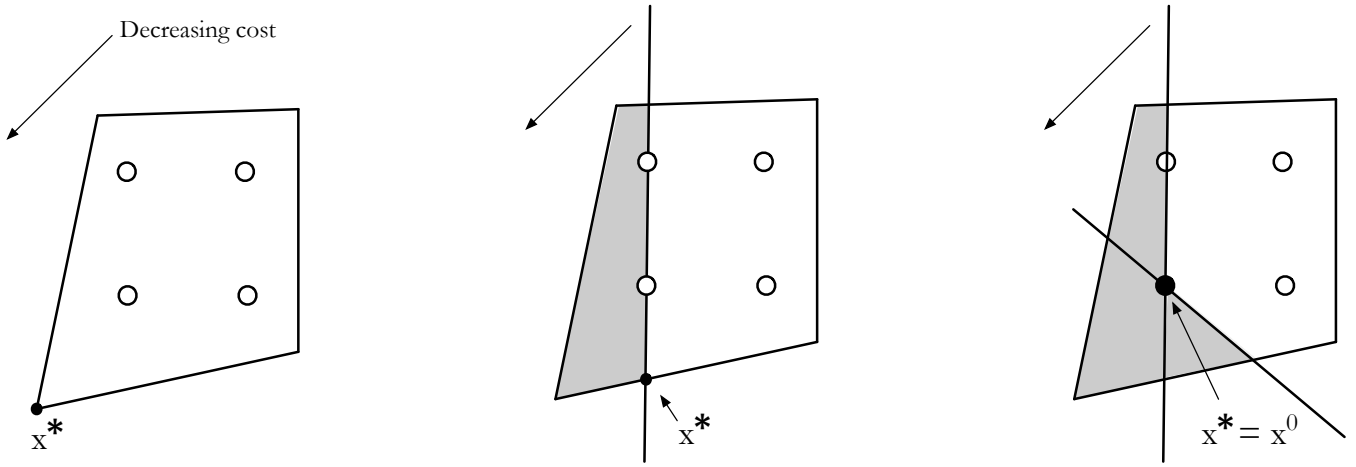
Integer Linear Programming (ILP)

- Using LP to Solve ILP?
  - If Solution is Integer, we're done.
  - Otherwise, $c(x^*)$ is lower/upper bound on $c(x^0)$
    - $c(x^*) \leq c(x^0)$ for minimization problem
    - $c(x^*) \geq c(x^0)$ for maximization problem
  - Provides Useful Information Still
    - If LP infeasible, so is ILP
    - If LP unbounded, so is ILP



Cutting-Plane Approach to ILP

- **Idea:**
  - Restrict feasible region without excluding any integer points
  - Add linear constraints one at a time until solution is integer
  - Use LP algorithm to find successive solutions



a) **b)** **c)**

Cutting-Plane Algorithm

- **Idea:**
  - Use one of the constraints to define a new constraint using integer values, thus removing the fractional part (the "Gomory" cut)
  - Add new constraint (increasing dimension of problem)
  - Solve it using the LP algorithm

- **Recall:** Standard Form
  - For some Basic variable x at the Boundary of the Polytope

$$x_{i\in\mathcal{B}} + \sum_{j\notin\mathcal{B}} a_{ij}\, x_j = b_i \quad \Rightarrow \quad x_{i\in\mathcal{B}} = b_i - \sum_{j\notin\mathcal{B}} a_{ij}\, x_j$$

Cutting-Plane Algorithm

- **Deriving an Integer Constraints:**
  - Given:

$$x_{i\in\mathcal{B}} + \sum_{j\notin\mathcal{B}} a_{ij}\, x_j = b_i \quad \Rightarrow \quad x_{i\in\mathcal{B}} = b_i - \sum_{j\notin\mathcal{B}} a_{ij}\, x_j$$

  - We can observe that:

$$\sum_{j\notin\mathcal{B}} \lfloor a_{ij} \rfloor\, x_j \leq \sum_{j\notin\mathcal{B}} a_{ij}\, x_j \quad \Rightarrow \quad x_{i\in\mathcal{B}} + \textstyle\sum_{j\notin\mathcal{B}} \lfloor a_{ij} \rfloor\, x_j \leq \lfloor b_i \rfloor$$

  - and thus the constraint with only integer values

$$\sum_{j\notin\mathcal{B}} (a_{ij} - \lfloor a_{ij} \rfloor) x_j \geq b_i - \lfloor b_i \rfloor$$

where $\lfloor v \rfloor$ is the integer part of v defined as the largest integer q that $q \leq v$

Cutting-Plane Algorithm

- **Deriving an Integer Constraints:**
  - Given:

$$x_{i\in\mathcal{B}} + \sum_{j\notin\mathcal{B}} a_{ij}\, x_j = b_i \quad \Rightarrow \quad x_{i\in\mathcal{B}} = b_i - \sum_{j\notin\mathcal{B}} a_{ij}\, x_j$$

  - We can observe that:

$$\sum_{j\notin\mathcal{B}} \lfloor a_{ij} \rfloor\, x_j \le \sum_{j\notin\mathcal{B}} a_{ij}\, x_j \quad \Rightarrow \quad x_{i\in\mathcal{B}} + \sum_{j\notin\mathcal{B}} \lfloor a_{ij} \rfloor\, x_j \le \lfloor b_i \rfloor$$

because $x_j \ge 0$

  - and thus the constraint with only integer values

$$\sum_{j\notin\mathcal{B}} (a_{ij} - \lfloor a_{ij} \rfloor) x_j \ge b_i - \lfloor b_i \rfloor$$

where $\lfloor v \rfloor$ is the integer part of v defined as the largest integer q that $q \le v$

Cutting-Plane Algorithm

- **Deriving an Integer Constraints (cont.):**
  - Similarly,

$$\sum_{j \notin \mathcal{B}} (a_{ij} - \lfloor a_{ij} \rfloor) x_j \geq b_i - \lfloor b_i \rfloor$$

  - and defining f_{ij} as the fractional part of a number, we get

$$f_{ij} = a_{ij} - \lfloor a_{ij} \rfloor \qquad \sum_{j \notin \mathcal{B}} f_{ij} x_j \geq f_i$$

  - And rewrite a new constraint in the Slack Form as:

$$s = -f_i + \sum_{j \notin \mathcal{B}} f_{ij} x_j$$

Properties of Integer Constraints

1. Cutting-Plane w.r.t. Optimal LP Solutions x^*

$$\sum_{j \notin \mathcal{B}} (a_{ij} - \lfloor a_{ij} \rfloor) x_j \geq b_i - \lfloor b_i \rfloor$$

That is, x* must violate at least one constraint as it is outside the Polytope defined by the integer constraint

Proof:

– This is trivially verified as

$$(b_i - \lfloor b_i \rfloor) \geq 0 \text{ and } x_{i \in N\mathcal{B}} = 0$$

where $\lfloor v \rfloor$ is the integer part of v defined as the largest integer q that $q \leq v$

Properties of Integer Constraints

2. It is satisfied by all integer feasible solution

- For all the integer points inside the polytope, this constraint is verified.

Proof:

- For each feasible solution of the linear problem, we have

$$\boldsymbol{x_{i\in\mathcal{B}} + \sum_{j\notin\mathcal{B}} \lfloor a_{ij} \rfloor x_j \;\leq\; x_{i\in\mathcal{B}} + \sum_{j\notin\mathcal{B}} a_{ij}\, x_j = b_i \quad x_j \geq 0} \quad (1)$$

- and in particular for each integer feasible solution:

$$\boldsymbol{x_{i\in\mathcal{B}} + \sum_{j\notin\mathcal{B}} \lfloor a_{ij} \rfloor x_j \;\leq \lfloor b_i \rfloor \quad x_j \geq 0} \text{ and integer} \quad (2)$$

- By subtracting (2) from (1) we get

$$\sum_{j\notin\mathcal{B}} (a_{ij} - \lfloor a_{ij} \rfloor) x_j \geq b_i - \lfloor b_i \rfloor$$

Cutting-Plane Approach to ILP

- Basic Cutting-Plane Algorithm:

```
Solve the non-integer LP max{c^T x: Ax = b, x ≥ 0}
and let x*_LP be an optimal basic feasible solution;
feasible = True;
WHILE x*_LP has fractional components AND feasible DO
  - Select a basic variable x_i with a fractional value;
  - Generate the corresponding Gomory cut;
  - Add the integer constraint to the problem;
  - Solve extended problem using an LP algorithm;
  IF problem is unbounded THEN
    feasible = False;
  END-IF
  set x*_LP as current feasible solution
END-WHILE
```

Cutting-Plane Approach to ILP

Theorem: If the ILP has a finite optimal solution, the Cutting Plane method finds one after adding a finite number of Gomory cuts.

- **Observations:**
  - Number of Cuts is Often very large
  - As we add Cuts, we increase the dimensionality of the LP problem to be solved (e.g., using the Simplex Algorithm)
  - Problems becomes increasingly more computationally expensive.

Cutting-Plane Example

ILP Problem:

Maximize $z = x_2$

subject to:

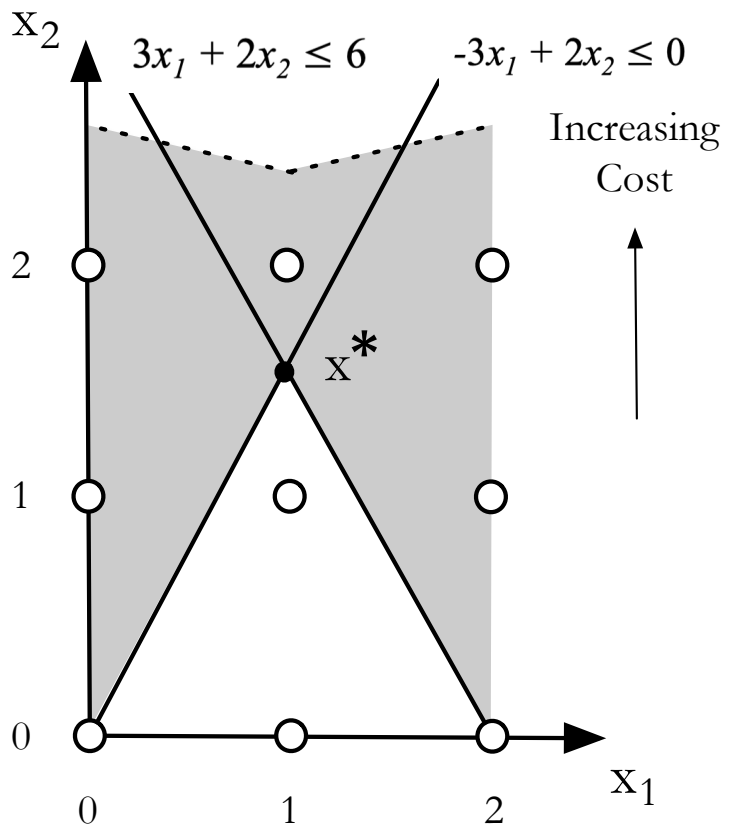
$3x_1 + 2x_2 \leq 6$

$-3x_1 + 2x_2 \leq 0$

x_1, x_2 integers ≥ 0

$\Downarrow$ *LP Solution*

$$
\begin{aligned}
z &= \frac{3}{2} - \frac{1}{4}x_3 - \frac{1}{4}x_4 \\
x_1 &= 1 - \frac{1}{6}x_3 + \frac{1}{6}x_4 \\
x_2 &= \frac{3}{2} - \frac{1}{4}x_3 - \frac{1}{4}x_4
\end{aligned}
$$



$\Rightarrow$ $x^* = (1, {}^3/_2) \Rightarrow z = {}^3/_2$

Cutting-Plane Example

First Cut:

From the constraint on x_2, the fractional basic variable, we get

$$\frac{1}{4}x_3 + \frac{1}{4}x_4 \leq {}^3/_2$$

fractional part to be used in the new constraint

which is equivalent to in the Slack form to

$$x_2 + \frac{1}{4}x_3 + \frac{1}{4}x_4 = {}^3/_2$$

$$\Downarrow$$

$$x_2 + 0\,x_3 + 0\,x_4 \leq \lfloor {}^3/_2 \rfloor \quad or \quad x_2 \leq 1$$

So, the constraint in Slack form is given by the equality below with a newly introduced variable, say x_5, we have

$$s = -f_i + \sum_{j \notin B} f_{ij}\, x_j \quad \Rightarrow$$

$$-\frac{1}{4}x_3 - \frac{1}{4}x_4 + x_5 = -{}^1/_2$$

$$\Downarrow \text{ replacing } x_3 \text{ and } x_4$$

$$x_5 = 1 - x_2$$

Cutting-Plane Example

First Cut:

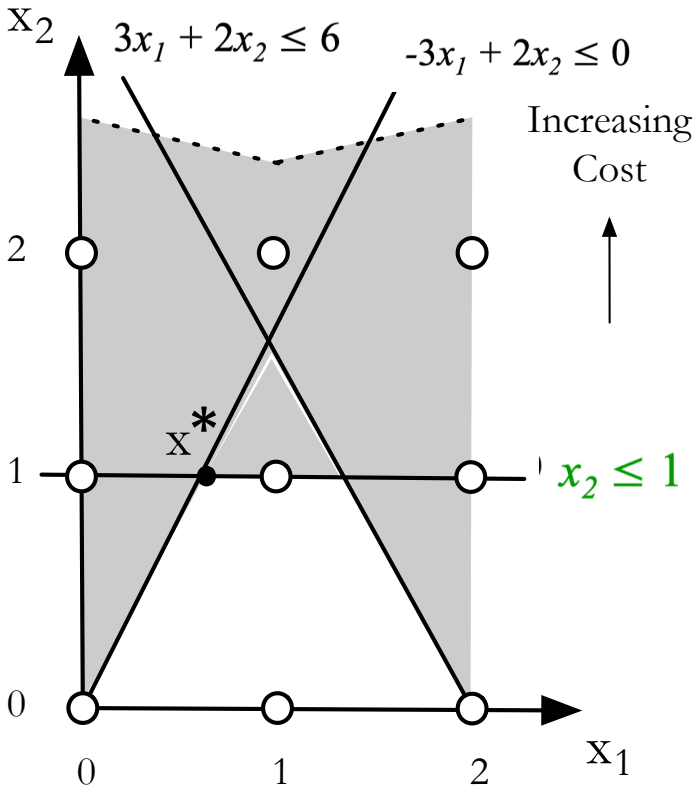
This results in the new LP problem:

$$
\begin{aligned}
z &= \frac{3}{2} + \frac{1}{4}x_3 + \frac{1}{4}x_4 \\
x_1 &= 1 + \frac{1}{6}x_3 - \frac{1}{6}x_4 \\
x_2 &= \frac{3}{2} - \frac{1}{4}x_3 - \frac{1}{4}x_4 \\
x_5 &= -\frac{1}{2} + \frac{1}{4}x_3 + \frac{1}{4}x_4
\end{aligned}
$$

$$\Downarrow$$

$$
\begin{aligned}
z &= 1 - x_5 \\
x_1 &= \frac{2}{3} + \frac{1}{3}x_4 - \frac{2}{3}x_5 \\
x_2 &= 1 - x_5 \\
x_3 &= 2 - x_4 + 4x_5
\end{aligned}
$$

$\Rightarrow$



$$x^* = (2/3, 1) \Rightarrow z = 1$$

Cutting-Plane Example

Second Cut:

From the constraint on x_1, the fractional basic variable, we get

$$x_1 - x_4 \leq \lfloor 2/3 \rfloor = 0$$

$$\Downarrow$$

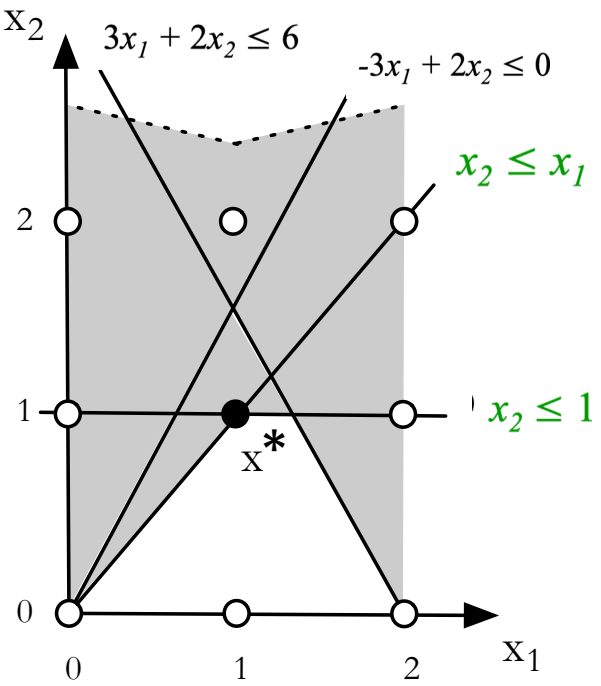
$$-2x_1 + 2x_2 \leq 0$$

$$\Downarrow$$

$$x_1 \geq x_2$$

$$\begin{aligned}
z &= 1 - x_5 \\
x_1 &= \frac{2}{3} + \frac{1}{3}x_4 - \frac{2}{3}x_5 \\
x_2 &= 1 + x_5 \\
x_3 &= 2 - x_4 + 4x_5 \\
x_6 &= -\frac{2}{3} + \frac{2}{3}x_4 + \frac{2}{3}x_5
\end{aligned}$$

$\Rightarrow$

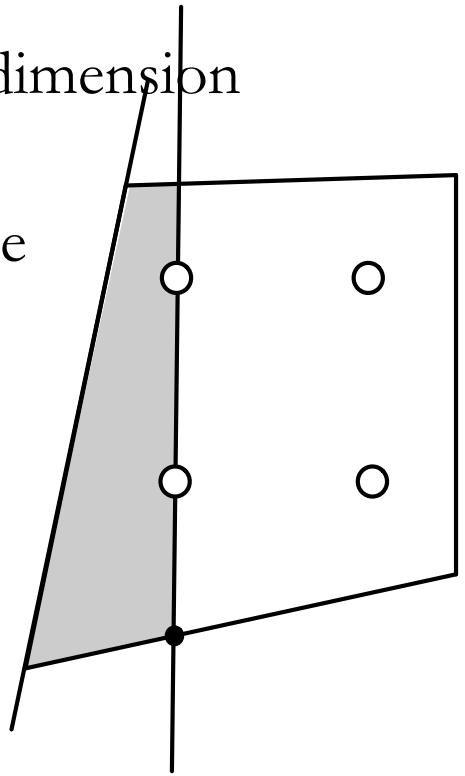


$x^* = x^0 = (1,1) \Rightarrow z = 1$

Integer Solution

Cutting-Plane Approach to ILP

- Relies on LP Algorithms
  - At each step adds a Constraint
  - Increases dimension of LP Formulation
  - "Forces" LP Solution to be Integer, one dimension at a time
  - Solution still found at a Vertex of Polytope



Branch-and-Bound Approach to ILP

- A popular and successful method of obtaining Integer Solutions is "Branch and Bound"
- A first step is to solve the problem as a LP problem, using a LP Method – e.g., the Simplex Algorithm
  - Produces Solutions with Variables in a continuous range;
  - Take one of these variables and constrain it to take the allowed (integer) **value below** its current value as well as the **value above** (also an integer value);
  - This defines 2 new LP problems (one for each value of this variable) which can be solved independently;
  - Repeat with another variable
- This explores a tree of LP problems

Branch-and-Bound Approach to ILP

- **Observations:**
  - Solutions with Variables in a continuous range are an Upper-Bound of Optimal Solutions
  - Solutions at Integer Feasible points are Lower-Bounds

- **Questions:**
  - What is the best way to sub-divide a given feasible region and which sub-divisions should be considered first?
  - Can the linear programs corresponding to the sub-divisions be solved efficiently?
  - Can the upper bound be improved?

Branch-and-Bound Example

- Example LP Problem:

Maximize $z = x_1 + x_2$

subject to:

$x_1 + 3x_2 \leq 3$

$3x_1 + x_2 \leq 6$

x_1, x_2 integers ≥ 0

The optimal solution to this LP problem is

$x_1 = 1.875$

$x_2 = 0.375$

$z = 2.25$

Try solving with constraint $x_2 \leq 0$

The new solution is

$x_1 = 2$

$x_2 = 0$

$z = 2$

Try solving with constraint $x_2 \geq 0$

The new solution is

$x_1 = 0$

$x_2 = 1$

$z = 1$

- Both of these solutions have given us integral values of x_1 also
- We choose the first solution because it has a higher z

Branch-and-Bound Example

- Example LP Problem:

Maximize $z = x_1 + x_2$

subject to:

$x_1 + 3x_2 \leq 3$

$3x_1 + x_2 \leq 6$

x_1, x_2 integers ≥ 0

The optimal solution to this LP problem is

$x_1 = 1.875$

$x_2 = 0.375$

$z = 2.25$

Try solving with constraint $x_2 \leq 0$

The new solution is

$x_1 = 2$

$x_2 = 0$

$z = 2$

Try solving with constraint $x_2 \geq 1$

The new solution is

$x_1 = 0$

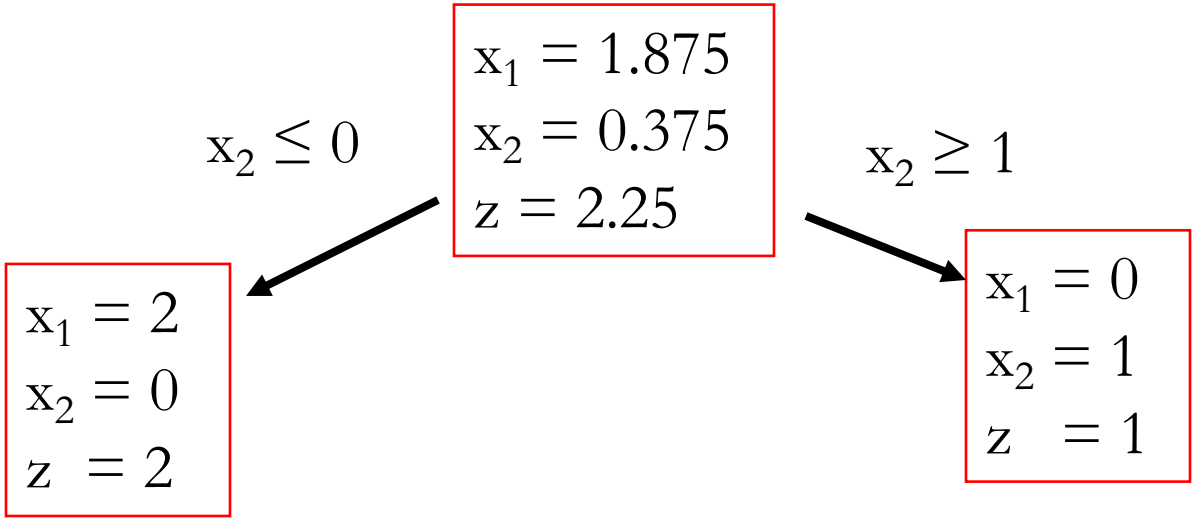
$x_2 = 1$

$z = 1$

- Both of these solutions have given us integral values of x_1 also
- We choose the first solution because it has a higher z

Branching Tree

- Overall: We explored a "branching tree" of LP problems and examined their solutions



- Procedure may need to go through several branches

Another Example

Maximize $z = 4x_1 + 6x_2 + 10x_3$

subject to:

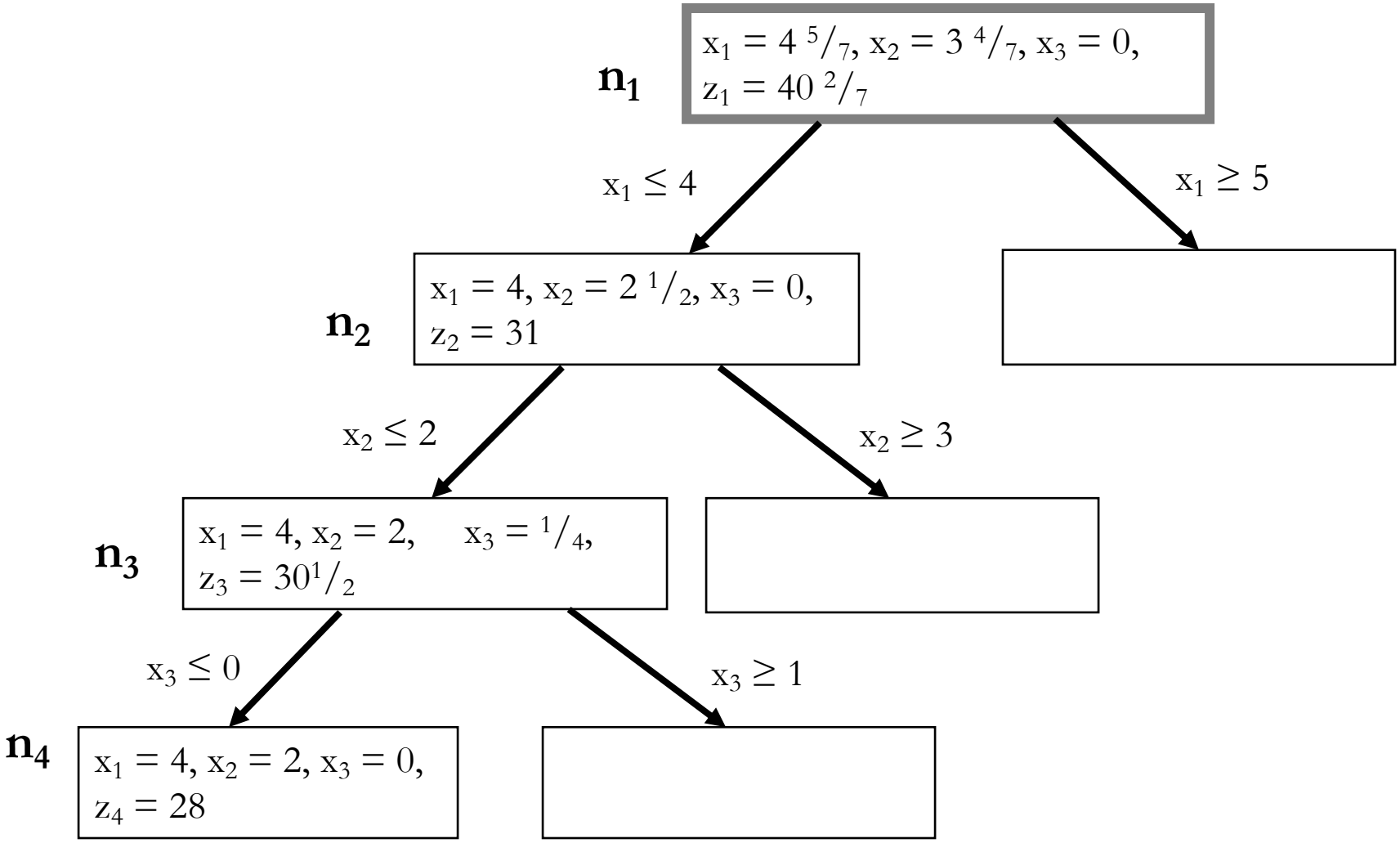
$2x_1 + x_2 + 2x_3 \leq 13$

$3x_1 - 2x_2 - 4x_3 \geq 7$

x_1, x_2, x_3 integers ≥ 0

We branch left (down) over the non-integer solutions with the lowest subscript

Another Example



Moving Down the Tree

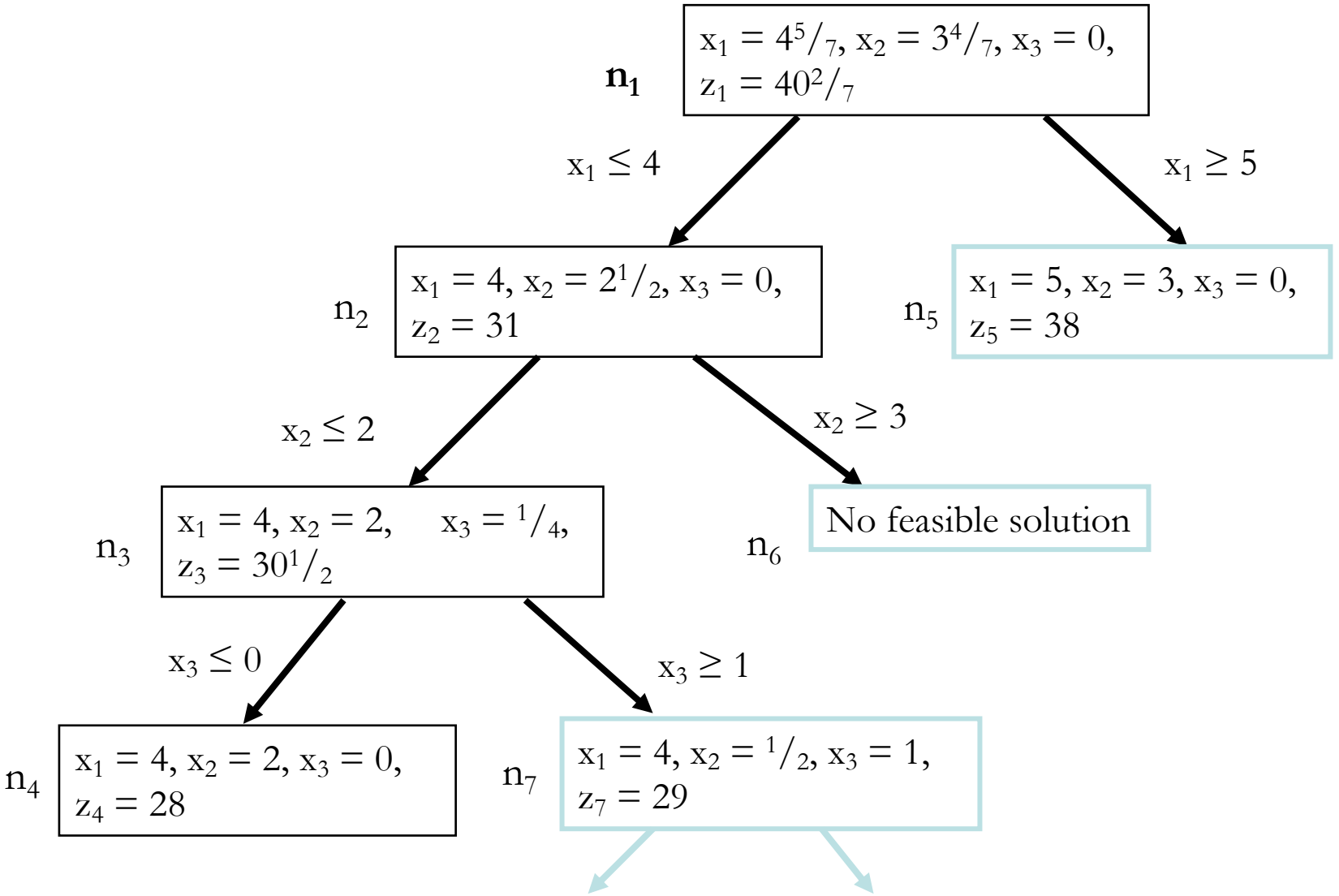
- In this tree, node $\mathbf{n_1}$ is the LP solution, and represents an upper bound on the objective function
- As we move through nodes $\mathbf{n_2}$, $\mathbf{n_3}$ and $\mathbf{n_4}$, we add more and more constraints on the variables
- As we move down, z never increases

Bound

- Node $\mathbf{n_4}$ is the first node found with an **all-integer** variables in the "solution" space
- Therefore, 28 is a lower bound on the optimal solution
- When we reach such a "solution", we have reached a "bound" in the branch and bound technique.
- We have not yet explored all the possibilities near the LP solution

- What to Do Next?
  - We now backtrack and explore some of the possibilities in the right hand branches

Bound

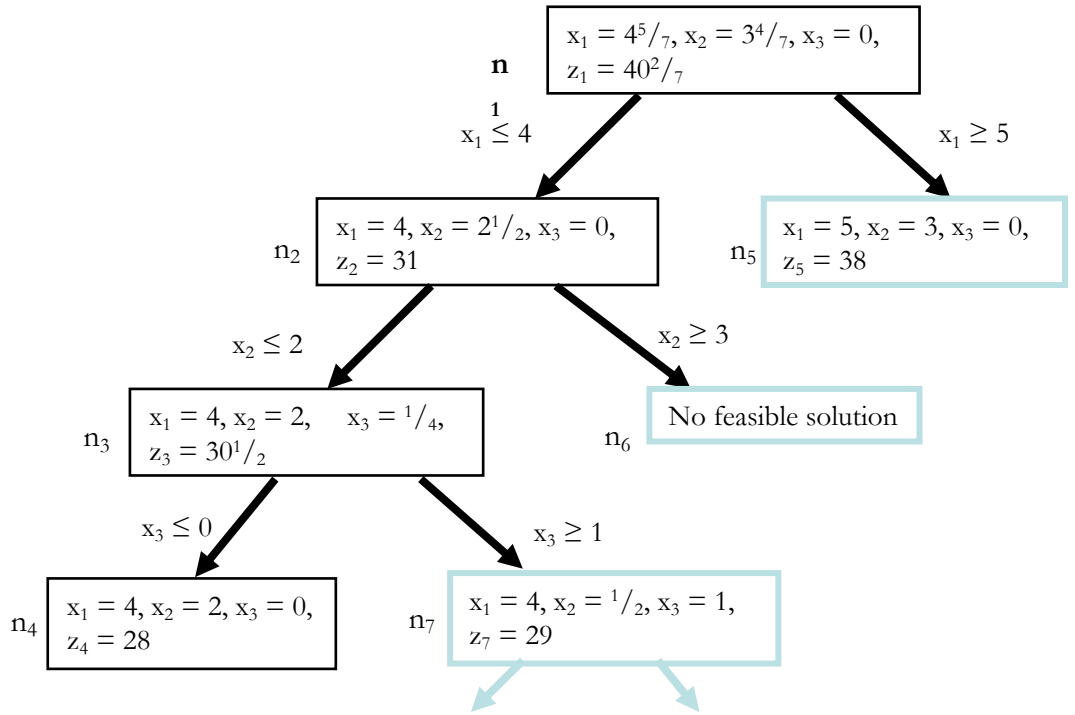

n1
x1 = 4 5/7, x2 = 3 4/7, x3 = 0,
z1 = 40 2/7
x1 ≤ 4
x1 ≥ 5
n2
x1 = 4, x2 = 2 1/2, x3 = 0,
z2 = 31
n5
x1 = 5, x2 = 3, x3 = 0,
z5 = 38
x2 ≤ 2
x2 ≥ 3
n3
x1 = 4, x2 = 2, x3 = 1/4,
z3 = 30 1/2
n6
No feasible solution
x3 ≤ 0
x3 ≥ 1
n4
x1 = 4, x2 = 2, x3 = 0,
z4 = 28
n7
x1 = 4, x2 = 1/2, x3 = 1,
z7 = 29

Further Branching

- Node n_5 has an **all-integer** solution, so we do not branch any further from this node (LP = ILP solution)
- Node n_6 does not have any feasible solutions: so no further branches
- Node n_7 does allow further branches…

Fathoming

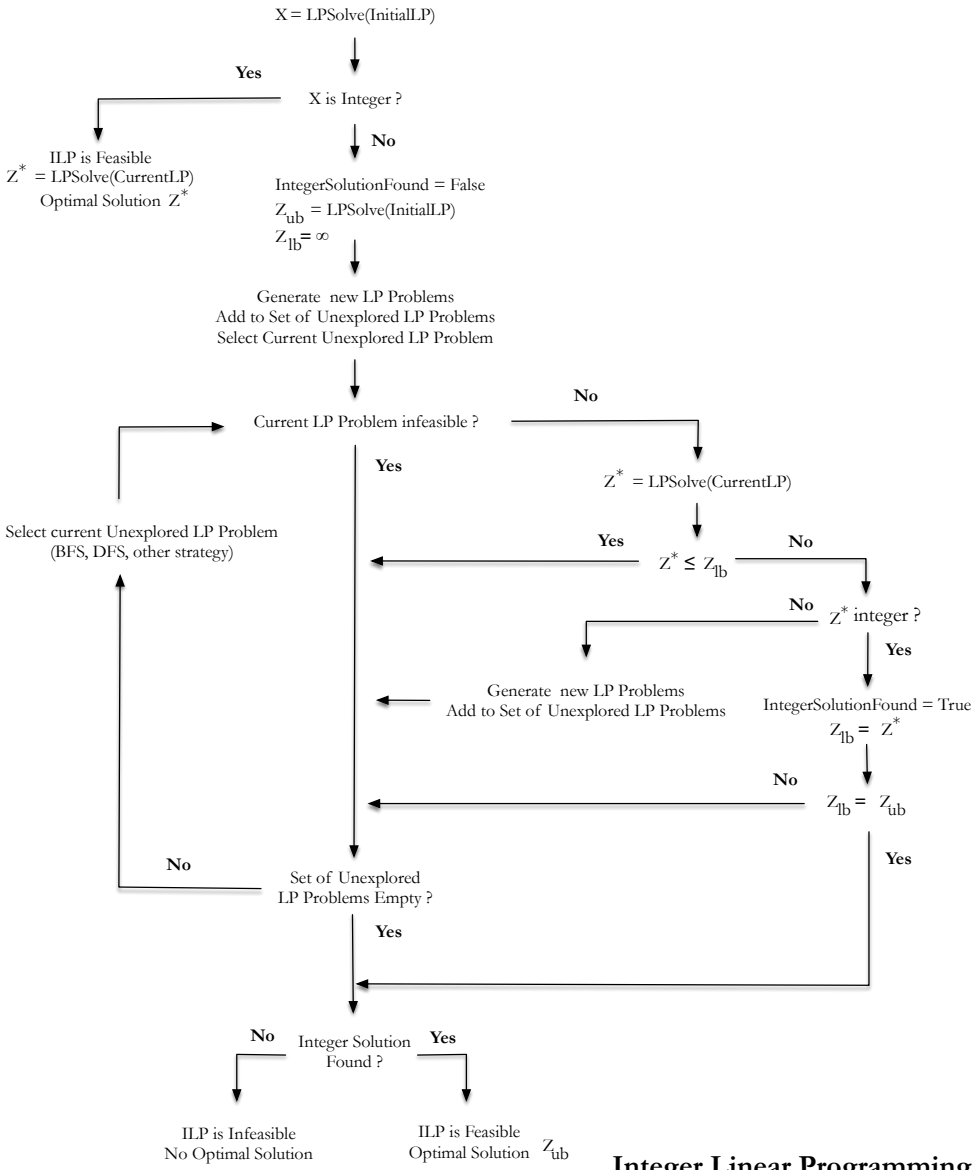
- Notice that $z_5 = 38$, and $z_7 = 29$
  - If we had evaluated node n_5 before n_7, we would not have needed to proceeded any further, since it would have been impossible for z_7 to have a higher objective function value than z_3, which is $30^1/_2$



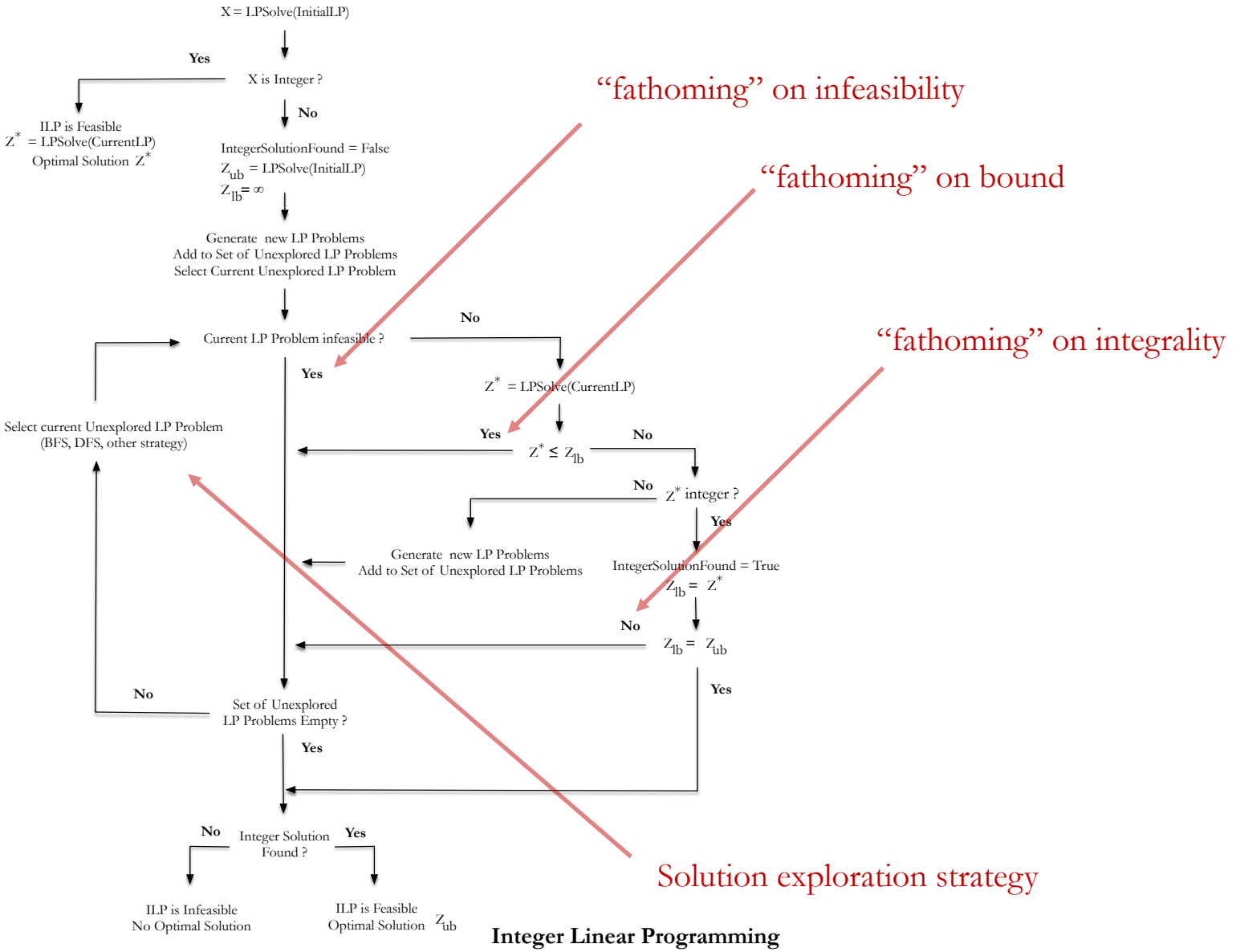
Fathoming

- Notice that $z_5 = 38$, and $z_7 = 29$
  - If we had evaluated node n_5 before n_7, we would not have needed to proceeded any further, since it would have been impossible for z_7 to have a higher objective function value than z_3, which is $30^1/_2$
- This is called "Fathoming"
- "Fathoming" Variants
  - Bounding: based on already found lower-bound
    - If found a non-integer solution with value lower than currently found integer bound, then skip exploration.
  - Integrality: found integer solution matches optimal (non-integral solution) solution
    - No need to explore further as other integer solutions will be worse.
  - Infeasibility: empty feasibility region.

Branch-and-Bound Algorithm



Branch-and-Bound Algorithm



Binary Tree

- The structure of solutions is a "binary tree".
- Every node in the tree represents a problem which must be solved by the LP method
- To reduce computing time, we try to keep the number of nodes in the tree as small as possible
- Strategies to Explore Tree:
  - Breath-First Search (BFS)
  - Depth-First-Search (DFS)
- Advantages:
  - Easily parallelizable, different processors computed different problems, sharing best results for bounding.

Solution Exploration Strategies

- Depth-First Strategy
  - The "depth first" strategy takes the left hand branch until the tree can be "pruned"
  - Then it backtracks to the last node where it could take another branch (to the right)
  - Then it continues with left branches
- Breath-First Strategy
  - Evaluates all nodes at level 2 before going to examine all nodes at level 3 and so on
- On Average…
  - It has been found that "depth-first" strategies are faster than "breadth first" strategies

LP/ILP Solvers

- CPLEX - proprietary IBM https://www.ibm.com/software
- MOSEK - proprietary http://www.mosek.com/
- GLPK - free http://www.gnu.org/software/glpk/
- GUROBI - proprietary http://www.gurobi.com/
- OR Tools (Google) https://developers.google.com/optimization

Summary

- Integer Linear Programming (ILP)
- Brute-Force and Naive Approaches
- Complexity
- The Cutting-Plane Method
- Branch-and-Bound Search